CS-781 Capstone, Phase 1: Legal Language Model Pre-training

Four architectures, a corpus audit, a cleaned 1B-token corpus, and retraining

Manish Kumar (251330604)
CS-781, IIT Kanpur

October 2026

Abstract

We train four substantially different legal language models (50–300M parameters) from scratch on the Phase-1 Indian legal corpus: a decoder-only GPT (A1), an encoder–decoder (A2), a prefix-LM with ALiBi (A3) and an efficient decoder with RoPE, grouped-query and sliding-window attention (A4). The first runs use the starter budget of about 200M tokens. An audit of that corpus found validation documents copied in the training split, duplicates, OCR word stutter and encoding residue, so we built a second data pipeline: whole-file sampling, character and markup normalisation, stutter repair, quality filters, exact and MinHash near-duplicate removal, a measured token budget, and a validation split that shares no text with either training corpus. We also made training 5.1× faster for A4 (3.6× for A1) with the same effective batch. On a validation set that neither corpus contains, retraining on the cleaned corpus at the same 200M-token budget lowers perplexity for every architecture (A4: 13.32 → 11.25), and continuing to 1B tokens lowers it further (A4: 8.26; A1: 8.37). Dev BERTScore F1 rises from 0.8269 to 0.8358 for A4. All numbers in this report are generated from the run outputs by `p1/report/make_figures.py`.

Contents

1 Introduction **3**

2 Data and tokenizer **3**
2.1 The Phase-1 corpus . . . 3
2.2 First data pipeline . . . 3
2.3 Tokenizer . . . 3

3 Methods **4**
3.1 Starter baseline (B0) . . . 4
3.2 Four architectures . . . 4
3.3 Training and evaluation setup . . . 4
3.4 Corpus audit . . . 5
3.5 Cleaned corpus . . . 5
3.6 Three validation sets . . . 7
3.7 Faster training . . . 7
3.8 Retraining protocol . . . 8

4 Experiments and results **8**
4.1 Original four architectures . . . 8
4.2 Cleaned corpus . . . 8
4.3 Throughput . . . 10
4.4 Retraining . . . 11
4.5 Evaluation on the three validation sets . . . 12

4.6 Test-set contamination 13
4.7 Use as a starting point for later phases 13

5 Analysis **13**

6 Limitations and next steps **14**

7 Conclusions **14**

8 Reproducibility **14**

1 Introduction

Phase 1 asks for a legal language model trained from scratch on the provided corpus of Acts, High Court, Supreme Court and Tribunal documents, which continues a 512-word legal prompt with 128 words. Submissions are scored with BERTScore F1 (primary), ROUGE-L F1 and BLEU-4. The rules fix the model size (50M–300M parameters), forbid pretrained weights and external corpora, require at least four substantially different architectures, and ask that training stay practical on free Colab/Kaggle-class hardware. The Phase-1 models are also the starting point for Phases 2–7, so their usefulness as encoders matters as much as their generation scores.

This report documents the whole Phase-1 work in the order it happened:

1. the starter-notebook baseline (Section 3.1);
2. a controlled comparison of four architectures on one data pipeline (Section 3.2, Section 3.3 and Section 4.1);
3. an audit of that data, which found nine problems (Section 3.4);
4. a cleaned corpus that addresses them (Section 3.5 and Section 4.2);
5. faster training without changing the optimisation (Section 3.7 and Section 4.3);
6. retraining all four architectures on the cleaned corpus, evaluated on three validation sets (Section 3.8, Section 4.4 and Section 4.5), and checks of test-set contamination (Section 4.6).

2 Data and tokenizer

2.1 The Phase-1 corpus

The corpus has four training sources (`train_acts.json` 0.7 GB, `train_supreme_court.json` 1.3 GB, `train_tribunals.json` 6.5 GB, `train_high_courts.json` 10.7 GB; more than 4.5B tokens in total) and a test file of 1,000 prompts. No validation split is provided.

2.2 First data pipeline

The first pipeline (`legal_lm/data/corpus.py`) streams each file with `ijson`, keeps a document with probability 0.5, collapses whitespace, drops documents under 50 words, sends 1% of the kept documents to validation, and stops reading a source once its share of the budget is filled. The mixing ratios are Acts 10%, Supreme Court 30%, High Courts 35%, Tribunals 25%; the budget is counted in words with an assumed 1.35 tokens per word. Table 1 shows what it produced: 187.5M training tokens and 4.96M validation tokens.

Table 1: First corpus (head-of-file sample). Tokens/word is measured with the legal BPE.

Source	Docs read	Train docs	Train words	Train tokens	Tokens/word
Acts	12,446	2,425	14.8M	18.7M	1.26
Supreme Court	55,528	12,163	44.4M	53.8M	1.21
High Courts	594,305	120,529	51.9M	66.2M	1.28
Tribunals	101,501	20,041	37.0M	48.8M	1.32
Total	763,780	155,158	148.2M	187.5M	1.27

2.3 Tokenizer

A byte-level BPE [15] with a 32,000-token vocabulary is trained on 100,000 documents of the training split (`scripts/02_train_tokenizer.py`). Byte-level BPE never produces unknown tokens (Devanagari names, section symbols and OCR debris all encode), and a legal vocabulary keeps

frequent words such as *respondent* or *Section* as single tokens: the measured fertility is 1.21–1.46 tokens per word depending on the source (Table 8), so a 512-word prompt fits in roughly 700 tokens of a 1,024-token context. The same tokenizer is used by every model in this report, and by Phases 2–7.

3 Methods

3.1 Starter baseline (B0)

The course notebook (`p1/p1_start.ipynb`) trains a randomly initialised GPT-2 model with the GPT-2 tokenizer: 8 layers, 8 heads, width 560, context 2,048, 59.5M parameters, 200M training and 5M validation tokens, one epoch, AdamW at 3×10^{-4} with cosine decay, about 0.63 GPU-hours at 88k tokens/s. It reached a validation loss of 2.733 (perplexity 15.38). Because the GPT-2 tokenizer splits legal text differently, its per-token perplexity is not comparable with the legal-BPE models below; it is kept as the starting reference only.

3.2 Four architectures

All four models share the tokenizer, data, token budget, schedule and evaluation, so that only the architecture changes (Table 2). Embeddings are tied with the output layer in every model.

Table 2: The four Phase-1 architectures (32k vocabulary, context 1,024).

ID	Design	Params
A1 GPT	Decoder-only [13]: learned absolute positions, pre-LayerNorm, GELU MLP, full causal multi-head attention. 8 layers, $d = 768$, 12 heads, $d_{\text{ff}} = 3072$.	82.1M
A2 enc-dec	Encoder–decoder [18]: bidirectional prompt encoder (6 layers) and causal decoder with cross-attention (6 layers), sinusoidal positions, ReLU MLP. $d = 640$, 10 heads, $d_{\text{ff}} = 2560$. Trained on a random split point: the encoder reads a prefix of $L/4$ to $L - 64$ tokens, the decoder predicts the rest.	89.4M
A3 prefix-LM	One stack whose mask makes the prompt bidirectional and the continuation causal (UniLM/GLM style [14]), ALiBi relative positions [12], no position embeddings. 8 layers, $d = 768$, 12 heads. With probability 0.5 an example is fully causal; otherwise a prefix of 64 to $0.75L$ tokens is read bidirectionally and only the rest is supervised.	81.3M
A4 efficient	Decoder with RoPE [17], RMSNorm [19], SwiGLU MLP [16] ($d_{\text{ff}} = 2048$), grouped-query attention [1] (12 query heads, 4 key/value heads), sliding-window attention of 256 tokens [2] with a global layer every third layer. 10 layers, $d = 768$.	87.5M

3.3 Training and evaluation setup

- **Optimisation.** AdamW [10] ($\beta = (0.9, 0.95)$, weight decay 0.1 on matrices only), peak learning rate 6×10^{-4}, 200 linear warm-up steps, cosine decay to 6×10^{-5}, gradient clipping at 1.0, bf16 autocast, no dropout.

- **Batch.** 8 sequences × 8 accumulation steps × 1,024 tokens = 65,536 tokens per optimizer step; 3,000 steps = 196.6M tokens, about one pass over the first corpus. Training windows are sampled at random positions of a memory-mapped `uint16` token file.

- **Validation and checkpoint selection.** Every 250 steps, 40 fixed batches of 1,025-token windows are scored: the first 768 tokens are context and the loss is the mean over the next 257 tokens, mirroring the task (continue a prompt). The checkpoint with the lowest validation loss is kept (`best.pt`); `last.pt` also stores the optimizer for resuming.

- **Generation.** Temperature 0.8, top-k 50, top-p 0.95 [5], repetition penalty 1.1 [6] over the last 256 tokens, end-of-text banned until 128 words are produced.
- **Generation metrics.** 200 local dev prompts (512 words + the true next 128 words) are cut from held-out documents; BERTScore F1 [20], ROUGE-L F1 [9] and BLEU-4 [11] are computed as on Codabench (`scripts/06_evaluate.py`).

3.4 Corpus audit

After the first runs we audited the training and validation splits (`scripts/11_audit_corpus.py`). Table 3 lists the problems, labelled A–I; each is addressed by one or more of the methods M1–M10 in Section 3.5 (mapping in Table 5).

Table 3: Problems found in the first corpus.

	Problem	Evidence
A	Under-trained base model	187.5M of >4.5B available tokens; about 20 tokens per parameter [4] would be ≈1.7B for an 87M model; validation loss still falling at the end of the schedule (Figure 2).
B	Validation leaks into train	29 validation documents are exact copies of training documents; at least 52 of 500 sampled validation documents share more than half their word 13-grams with a 30k-document training sample.
C	No deduplication	1,090 exact duplicate training documents (0.7%); 1,176 extra copies of the same first 300 characters; one Bombay HC form header occurs 28 times.
D	OCR word stutter	566 documents with five or more tripled words ("Parekh Parekh Parekh Petitioner Petitioner Petitioner"), about 2.4M words.
E	Encoding / markup noise	About 70 documents with the replacement character U+FFFD, RTF control words (`\par \pard \tab`), 513 Acts documents with runs of 20+ dots or dashes.
F	Short template orders	32% of High Court documents have fewer than 200 words (median 291 words).
G	Token budget missed	Budget estimated at 1.35 tokens/word; measured 1.21–1.32, so training got 187.5M instead of 200M tokens.
H	Head-of-file sampling	Each file is read from its first record until the budget fills, so documents later in a file are never seen.
I	Long-range downstream labels	(Phase 2) party roles are declared in the header and must be resolved thousands of tokens later; recorded here because it motivates longer context.

3.5 Cleaned corpus

The second pipeline is `legal_lm/data/clean.py`, enabled by `configs/clean.yaml` (Table 4). Figure 1 shows the order of the steps.

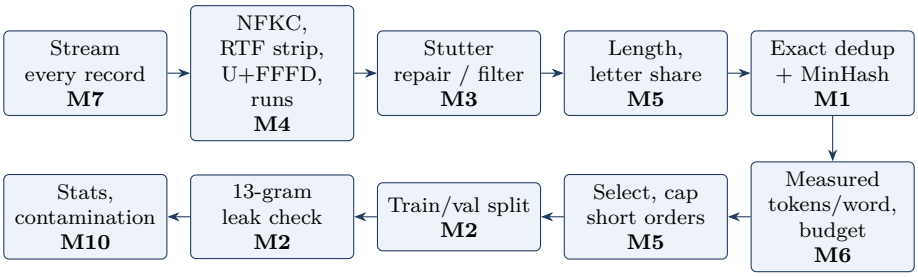


Figure 1: Cleaned-corpus pipeline. Steps M1–M5 run in a 24-process pool over every record of the four files.

M7 Whole-file sampling. Every record of every file is read and cleaned, instead of the head of each file, so the selection in M6 samples uniformly from the whole source. Reading is throttled (at most 96 batches of 64 documents in flight) so a 10.7 GB file streams in bounded memory.

M4 Character and markup normalisation. Documents containing U+FFFD are dropped (the original character is lost). Text is normalised with Unicode NFKC (ligatures, full-width forms). When a document has at least three RTF control words, all RTF control words, hex escapes and braces are removed. Whitespace is collapsed (the test prompts are word sequences), spaced dot leaders become "...", and runs of four or more identical `.` `-` `_` `=` `*` `~` characters are cut to three.

M3 OCR stutter repair. A unit of one to four words repeated three or more times in a row, containing at least two letters, is replaced by one copy ("CIVIL WRIT WRIT WRIT PETITION" → "CIVIL WRIT PETITION"); numbers and punctuation runs are left alone, and doubled words ("that that") are kept. If the repairs removed more than 2% of a document's words, the document is dropped, since its text is unreliable beyond what the rule can fix.

M5 Quality filters and source caps. A per-source minimum length (100 words for High Courts, 50 otherwise) and a minimum share of letters (60% of characters). Within High Courts, documents under 200 words may hold at most 25% of the source's tokens.

M1 Exact and near-duplicate removal. The exact key is the MD5 of the lower-cased word sequence; repeats are dropped as they stream. For near duplicates each document gets a 128-permutation MinHash signature [3] over its word 5-grams (multiply–shift hashing of 64-bit polynomial shingle hashes). Locality-sensitive hashing with 16 bands of 8 rows [8] proposes candidate pairs (collision probability rises steeply around Jaccard 0.7); a pair is accepted only if at least 80% of the 128 signature values agree. Accepted pairs are merged with union–find and the longest document of each cluster is kept. Deduplication follows the evidence that duplicated training text hurts language models and inflates validation scores [7].

M6 Measured token budget. For each source, 2,000 surviving documents are tokenized with the real BPE to measure tokens per word; the budget (1B training + 5M validation tokens) is then allocated by the mixing ratios in tokens. A source with fewer tokens than its share contributes everything it has and its deficit is redistributed to the others (Supreme Court had only 248M tokens after cleaning). Within a source, documents are taken in random order; a document that would overshoot the remaining budget by more than 1% is skipped.

M2 Leak-free split. Because near-duplicate clusters were already collapsed, no cluster can straddle the split. The split is then checked: every validation candidate whose word 13-grams overlap the new training split *or the first corpus* by more than 50% is dropped. The 13-grams are hashed and only hashes divisible by 8 are kept, the same subset in every document, which makes the check affordable on 1B tokens. Checking against the first corpus as well means the old models (R0) are also scored on unseen text.

M10 Statistics and contamination report. Every counter (documents read, dropped per reason, selected, leaked) is written to `data_clean/corpus/stats.json`. The same 13-gram test measures how much of the Phase-1 test prompts and the Phase-2 test documents appear in the training text (report only; nothing is removed).

Table 4: Cleaning settings (`configs/clean.yaml`).

Method	Setting	Value
M4	RTF stripping when control words $\geq$	3
M3	Maximum share of words removed by stutter repair	2%
M5	Minimum words (High Courts / others)	100 / 50
M5	Minimum letter share	0.6
M5	Short-order cap (High Courts, docs <200 words)	25% of tokens
M1	Shingle length / MinHash permutations / LSH bands	5 words / 128 / 16×8
M1	Signature agreement to accept a near duplicate	0.8
M6	Documents per source to measure tokens/word	2,000
M2	Leak n-gram / kept fraction / maximum overlap	13 words / 1 in 8 / 0.5

Table 5: Which method addresses which problem.

	M1	M2	M3	M4	M5	M6	M7	retrain
A under-trained						•	•	•
B leakage	•	•						
C duplicates	•							
D OCR stutter			•		•			
E encoding / markup				•	•			
F short orders	•				•			
G budget						•		
H head-of-file							•	

3.6 Three validation sets

- **Old val**: the first corpus's validation split (4.96M tokens). It overlaps the first training split (problem B), so it favours the original models.
- **Leak-free val**: the cleaned corpus's validation split (4.10M tokens), drawn from the same whole-file distribution as the new training data and checked against both training corpora.
- **Fair val**: the old validation documents that overlap *neither* training corpus by more than 50% (`scripts/10_fair_val.py`): 1,990 of 3,764 documents, 2.04M tokens. It has the old (head-of-file) distribution and is unseen by every model, so it is the fairest old-versus-new comparison. It contains no Supreme Court documents: the cleaned corpus uses every Supreme Court document, so all old Supreme Court validation documents are in the new training split.

3.7 Faster training

Two changes leave the optimisation unchanged and only alter how each step is computed:

- **Micro-batch / accumulation split.** 16 sequences × 4 accumulation steps (A2: 32 × 2) instead of 8 × 8: the same 65,536 tokens per optimizer step and the same gradient up to floating-point order, but fewer, larger kernel launches.
- `torch.compile` for A1, A3 and A4, which fuses the element-wise operations of each block. A2 cannot be compiled: its encoder length changes every step (random split point), and the compiler fails on the resulting dynamic shapes.

`PYTORCH_CUDA_ALLOC_CONF=expandable_segments` could not be used on this machine (it needs NVML, which is broken here).

3.8 Retraining protocol

- **R0**: the original checkpoints (Section 4.1).
- **R1**: each architecture trained from scratch on the cleaned corpus with exactly the original schedule: 3,000 steps, 196.6M tokens. Only the data changes, so R1 vs R0 measures the cleaning.
- **R2**: R1's last weights continued for 12,260 more steps (803.5M tokens), 1B tokens seen in total. The optimizer is fresh and the schedule restarts: 200 warm-up steps to the same peak 6×10^{-4}, cosine decay to 6×10^{-5} (`configs/clean_r2.yaml`, `train.init_from`). R2 vs R1 measures more data.

Two queues ran in parallel on one shared H200 (A4 then A1; A3 then A2), since four concurrent runs did not fit next to another tenant's server (`run_clean.sh`).

4 Experiments and results

4.1 Original four architectures

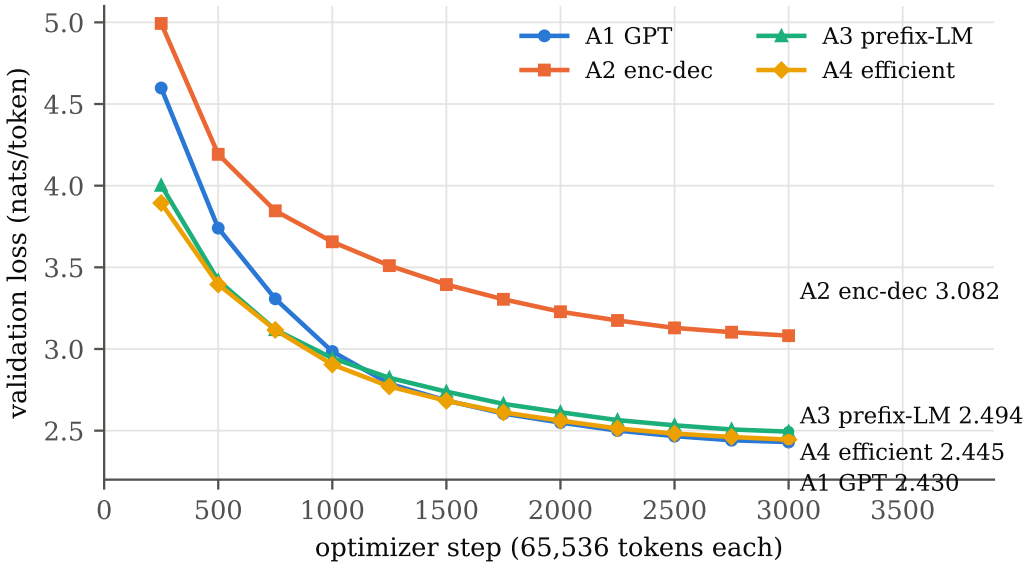


Figure 2: Validation loss of the original runs (first corpus, old val). Final values are labelled.

Table 6: Original Phase-1 runs (old val; dev metrics on the first dev set).

Model	Params	Non-emb.	Val loss	Val ppl	BERTScore	ROUGE-L	BLEU-4	GPU-h
A1 GPT	82.1M	56.7M	2.430	11.36	0.8364	0.2225	0.1104	0.39
A2 enc-dec	89.4M	68.9M	3.082	21.79	0.8154	0.1732	0.0318	0.87
A3 prefix-LM	81.3M	56.7M	2.494	12.11	0.8349	0.2202	0.1100	0.41
A4 efficient	87.5M	62.9M	2.445	11.53	0.8346	0.2234	0.1104	0.75

4.2 Cleaned corpus

Table 7 counts every document of the four files by outcome and Figure 3 ranks the reasons. In total 3,480,220 documents were read; 106,717 were exact duplicates, 204,246 near duplicates (1,688,774 verified pairs), 90,856 were dropped for heavy OCR stutter and 125,397 were repaired and kept (2.34M words removed).

Table 7: Documents per source by outcome of cleaning (before budget selection).

Source	Read	U+FFFD	Stutter	Too short	Few letters	Exact dup.	Near dup.	Available
Acts	18,822	3	31	5	68	124	1,016	17,575
Supreme Court	55,528	38	3	87	29	161	972	54,238
High Courts	2,536,738	0	90,585	376,477	5,324	101,433	158,426	1,804,493
Tribunals	869,132	487	237	3,244	23,972	4,999	43,832	792,361
Total	3,480,220	528	90,856	379,813	29,393	106,717	204,246	2,668,667

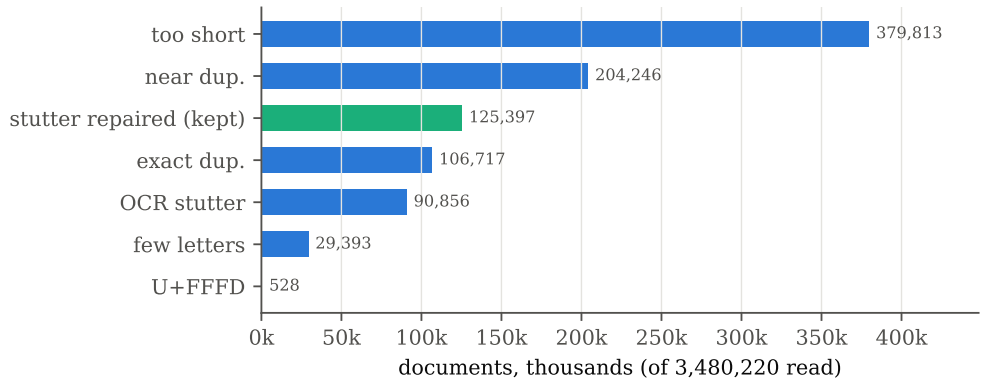


Figure 3: Documents removed by each rule (blue) and documents repaired by the stutter rule and kept (green).

Table 8: Budget selection and split. Target = token share by mixing ratio after redistribution (M6).

Source	Tokens/word	Target	Train tokens	Train docs	Val docs	Val dropped (leak)
Acts	1.260	108.1M	107.7M	13,577	37	29
Supreme Court	1.217	248.0M	246.5M	53,995	181	62
High Courts	1.355	378.5M	380.5M	480,572	1,859	523
Tribunals	1.455	270.3M	271.3M	178,353	806	150
Total		1005.0M	1006.0M	726,497	2,883	764

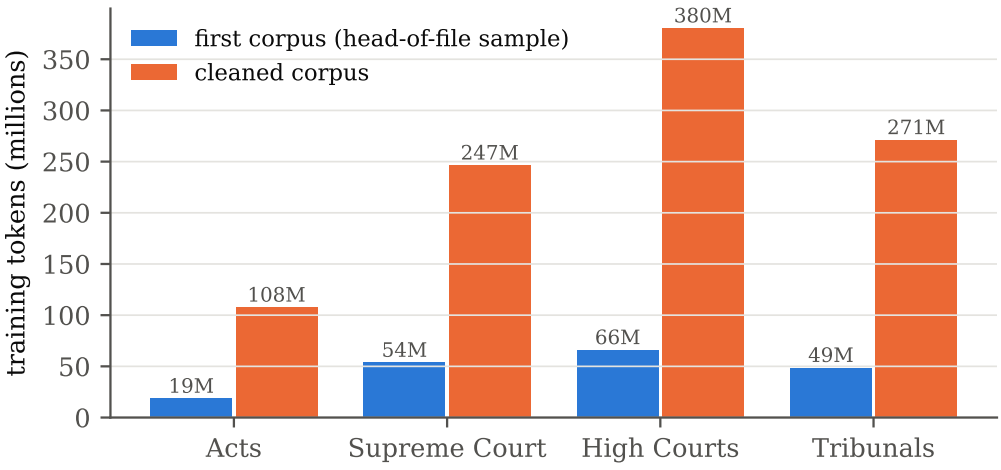


Figure 4: Training tokens per source in the first and the cleaned corpus.

Audit of the cleaned corpus (M10)

Table 9 reruns the audit of Section 3.4 (`scripts/11_audit_corpus.py`) on both corpora. The cleaned corpus is 4.7 times larger, so rates are given next to counts. Reading every record (M7) also brought in text the first corpus never reached: 22,774 Tribunal documents (12.8% of the source) contain Devanagari and 841 are more than 20% non-ASCII, and many High Court and Tribunal documents carry URLs. These were not targeted by any cleaning rule.

Table 9: Audit before and after cleaning (counts, share of training documents).

Check	First corpus	Cleaned corpus
Training documents	155,158	726,497
Exact duplicate training docs	1,090 (0.70%)	0 (0.00%)
Extra docs sharing first 300 chars	1,176 (0.76%)	1,708 (0.24%)
Val docs copied in train	29	0
Val docs >50% 13-gram overlap (sample)	52/500	0/500
Docs with ≥5 stutter triples	566 (0.36%)	25 (<0.01%)
Docs with U+FFFD / mojibake	70 (0.05%)	163 (0.02%)
Docs with 20+ dot/dash runs	983 (0.63%)	238 (0.03%)
Docs containing Devanagari	96 (0.06%)	23,117 (3.18%)
Docs >20% non-ASCII	0 (0.00%)	842 (0.12%)
Docs under 200 words	40,092 (25.84%)	145,273 (20.00%)
Docs containing URLs	82 (0.05%)	22,513 (3.10%)

4.3 Throughput

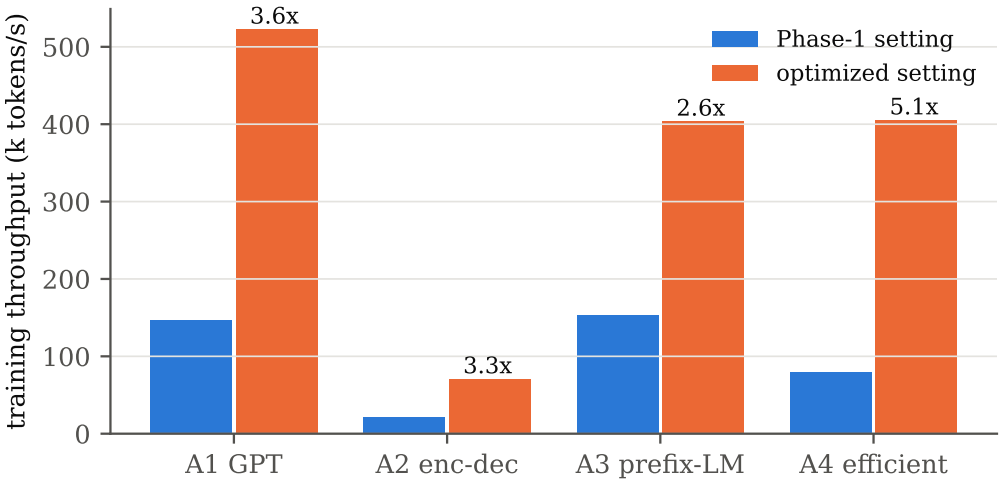


Figure 5: Training throughput before and after the changes of Section 3.7. Benchmarks of 40–60 steps on the H200 while the corpus build was using the CPUs, so absolute numbers are lower than in an idle machine; the ratios are the point.

Table 10: Throughput benchmarks (tokens/s).

Model	Settings measured	Speed-up
A1 GPT	base (mb 8, eager): 147k; optimized (mb 16, compile): 523k	3.6×
A2 enc-dec	base (mb 8, eager): 22k; optimized (mb 32, eager): 71k	3.3×
A3 prefix-LM	base (mb 8, eager): 153k; optimized (mb 16, compile): 404k	2.6×
A4 efficient	base (mb 8, eager): 79k; compile only: 251k; optimized (mb 16, compile): 405k	5.1×

4.4 Retraining

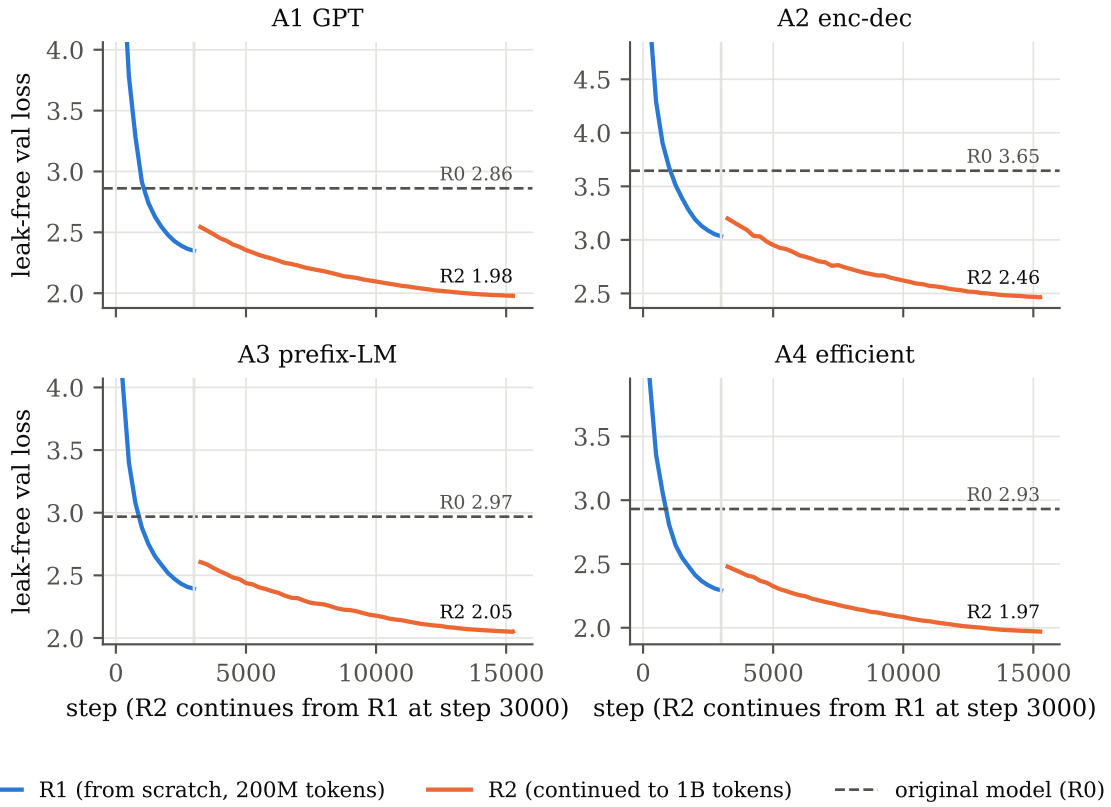


Figure 6: Leak-free validation loss of R1 (steps 0–3,000) and R2 (continued from R1; the restarted warm-up shows as the jump at step 3,000). The dashed line is the original model (R0) on the same set.

Table 11: Retraining runs. Throughput is the median over logged steps with two runs sharing the GPU.

Model	Run	Steps	Tokens	Median tok/s	GPU-h	Peak alloc. (GB)
A1 GPT	R1	3,000	197M	265k	0.22	11.7
	R2	12,260	803M	266k	0.80	11.8
A2 enc-dec	R1	3,000	197M	266k	0.29	16.9
	R2	12,260	803M	273k	0.97	16.9
A3 prefix-LM	R1	3,000	197M	225k	0.25	11.7
	R2	12,260	803M	224k	1.02	11.7
A4 efficient	R1	3,000	197M	203k	0.25	12.9
	R2	12,260	803M	202k	1.12	12.9

4.5 Evaluation on the three validation sets

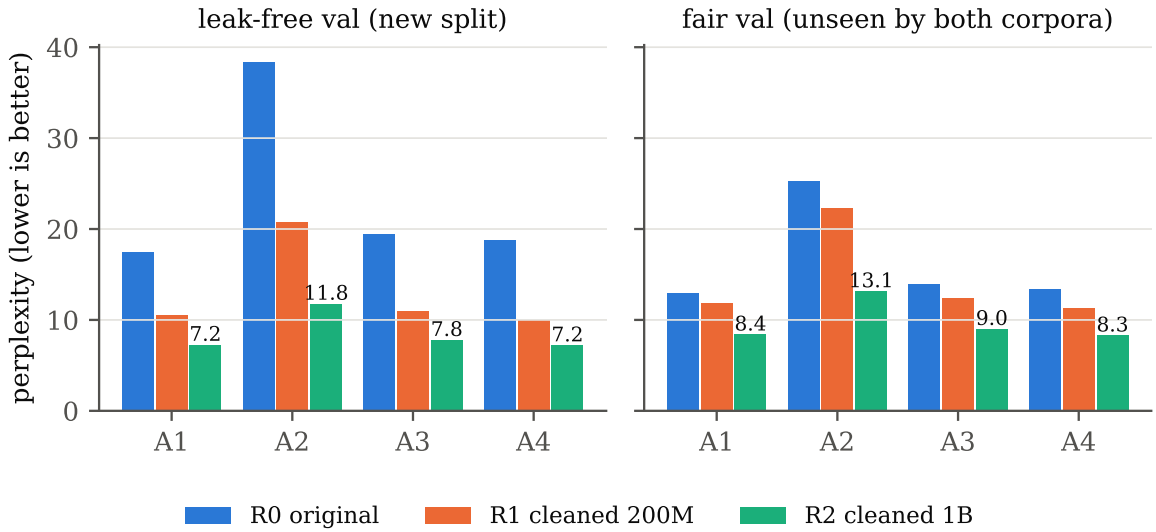


Figure 7: Perplexity of R0, R1 and R2 on the leak-free and the fair validation sets (R2 values labelled; all values in Table 12).

Table 12: Perplexity on the three validation sets and generation metrics on the cleaned dev prompts (200 prompts from leak-free validation documents; R0 regenerated on the same prompts).

		Perplexity			Dev generation		
Model	Run	Old val	Leak-free	Fair	BERTScore	ROUGE-L	BLEU-4
A1 GPT	R0	11.36	17.49	12.93	0.8286	0.1920	0.0582
	R1	–	10.47	11.82	0.8292	0.1988	0.0736
	R2	–	7.23	8.37	0.8359	0.2137	0.0938
A2 enc-dec	R0	21.79	38.32	25.26	0.8170	0.1725	0.0275
	R1	–	20.78	22.31	0.8162	0.1710	0.0291
	R2	–	11.75	13.12	0.8245	0.1844	0.0398
A3 prefix-LM	R0	12.11	19.46	13.89	0.8255	0.1851	0.0473
	R1	–	10.95	12.42	0.8277	0.1941	0.0596
	R2	–	7.76	8.98	0.8341	0.2092	0.0877
A4 efficient	R0	11.53	18.75	13.32	0.8269	0.1861	0.0502
	R1	–	9.91	11.25	0.8325	0.2045	0.0796
	R2	–	7.17	8.26	0.8358	0.2127	0.0928

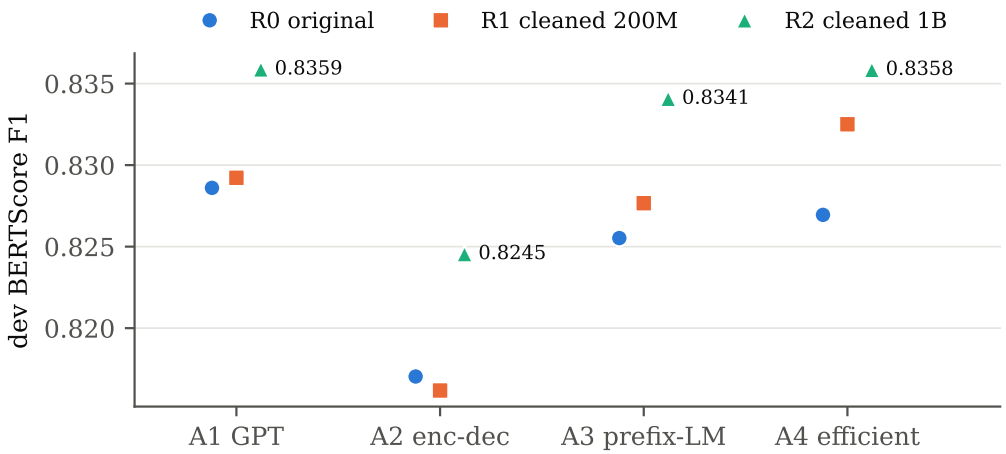


Figure 8: Dev BERTScore F1 on the cleaned dev prompts (R2 values labelled).

On the old validation set, which favours the original models, A4 R1 scores a perplexity of 10.18 against 11.53 for the original A4 (`scripts/09_eval_val.py`).

4.6 Test-set contamination

With the 13-gram test of M10 (a document counts if more than half of its sampled 13-grams occur in the reference text):

- 226 of the 1,000 Phase-1 test prompts and 26 of the 35 Phase-2 test documents overlap the training text (cleaned training split plus the first corpus) by more than 50%.
- Against each corpus alone: Phase-2 test documents with >90% overlap, 5 of 35 for the first corpus and 25 of 35 for the cleaned corpus; Phase-2 training documents, 8 of 70 and 50 of 70.

The test documents are part of the official training files, so reading every record (M7) includes them. Nothing was removed; this is reported so that later phases can be interpreted with it in mind.

4.7 Use as a starting point for later phases

The Phase-1 checkpoints were reused as backbones. Phase 2 (legal NER, A4 + LoRMA, mean macro-F1 over three dev splits): original 0.488, R1 0.479, R2 0.467. Phase 3 (rhetorical roles, A4 + LoRMA, out-of-fold macro-F1 over 90 documents): original 0.427, R2 0.455.

5 Analysis

To be written by the author — analysis of the results

The course rules (§2, item 9) do not allow LLM-written result analysis. Questions the course asks the report to answer (§7.4, §20.1), with the evidence above:

- Which architecture works best under the same budget, and why (Figure 2 and Table 6)?
- How did data quality and data quantity each affect the models (R0 vs R1 vs R2, Table 12)?
- How much did the leaky validation split change the conclusions one would draw (old vs fair val)?
- What did the throughput changes cost or save (Table 10 and Table 11)?
- Why might a lower perplexity not transfer to every downstream task (Section 4.7)?
- What failed, and why (e.g. A2's encoder output, `torch.compile` on A2, four concurrent runs)?

6 Limitations and next steps

Facts that limit the conclusions (the interpretation is the author's):

- One seed per run; no confidence intervals on perplexity or BERTScore.
- Decoding was not tuned on the new models (one setting, Section 3.3).
- No Phase-1 compression experiment was run on the cleaned models (a distillation config exists: `configs/distill_efficient_small.yaml`).
- The cleaning thresholds (Table 4) were set once, not ablated individually; R1 measures all of M1–M7 together.
- Downstream test documents occur in the cleaned training text (Section 4.6).
- Throughput benchmarks ran on a shared machine under CPU load.
- The cleaned corpus contains Hindi/Devanagari Tribunal documents and URLs (Table 9), which no rule filtered; their share and effect were not measured separately.
- Template headers remain: Table 9 still finds documents sharing their first 300 characters (different bodies, so not near duplicates).

To be written by the author — what you would do next
With more time or compute: ...

7 Conclusions

To be written by the author — conclusions
Summarise what you conclude from the experiments, in your own words.

8 Reproducibility

Code: `capstone_llm/` (Phase 1).

To be written by the author — repository link
GitHub: `https://github.com/...`

First pipeline	`bash run_all.sh`
Cleaned corpus	`python scripts/01_prepare_data.py -config configs/base.yaml configs/clean.yaml`
Tokenize	`python scripts/03_tokenize.py -config configs/base.yaml configs/clean.yaml`
R1 + R2, all models	`bash run_clean.sh`
R0 on another val set	`python scripts/09_eval_val.py -val data_clean/tokens/val.bin -ckpt ...`
Fair val	`python scripts/10_fair_val.py`
Corpus audit	`python scripts/11_audit_corpus.py data_clean/corpus`
Figures and tables	`python p1/report/make_figures.py`

LLM declaration

To be written by the author — LLM declaration (mandatory)
Fill in the course questionnaire. Describe honestly which parts used an LLM assistant (for example code for the cleaning pipeline, scripts, figures and this report's methods/results text) and which parts are your own (design decisions, hyperparameters, analysis, conclusions).

References

[1] J. Ainslie et al. GQA: Training generalized multi-query transformer models from multi-head checkpoints. EMNLP 2023.

[2] I. Beltagy, M. Peters, A. Cohan. Longformer: The long-document transformer. arXiv:2004.05150, 2020.

[3] A. Broder. On the resemblance and containment of documents. Compression and Complexity of Sequences, 1997.

[4] J. Hoffmann et al. Training compute-optimal large language models. NeurIPS 2022.

[5] A. Holtzman et al. The curious case of neural text degeneration. ICLR 2020.

[6] N. Keskar et al. CTRL: A conditional transformer language model for controllable generation. arXiv:1909.05858, 2019.

[7] K. Lee et al. Deduplicating training data makes language models better. ACL 2022.

[8] J. Leskovec, A. Rajaraman, J. Ullman. Mining of Massive Datasets, ch. 3. Cambridge University Press, 2014.

[9] C.-Y. Lin. ROUGE: A package for automatic evaluation of summaries. ACL Workshop 2004.

[10] I. Loshchilov, F. Hutter. Decoupled weight decay regularization. ICLR 2019.

[11] K. Papineni et al. BLEU: a method for automatic evaluation of machine translation. ACL 2002.

[12] O. Press, N. Smith, M. Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. ICLR 2022.

[13] A. Radford et al. Language models are unsupervised multitask learners. OpenAI, 2019.

[14] C. Raffel et al. Exploring the limits of transfer learning with a unified text-to-text transformer. JMLR 2020.

[15] R. Sennrich, B. Haddow, A. Birch. Neural machine translation of rare words with subword units. ACL 2016.

[16] N. Shazeer. GLU variants improve transformer. arXiv:2002.05202, 2020.

[17] J. Su et al. RoFormer: Enhanced transformer with rotary position embedding. arXiv:2104.09864, 2021.

[18] A. Vaswani et al. Attention is all you need. NeurIPS 2017.

[19] B. Zhang, R. Sennrich. Root mean square layer normalization. NeurIPS 2019.

[20] T. Zhang et al. BERTScore: Evaluating text generation with BERT. ICLR 2020.